Universidad Autónoma de Aguascalientes
Centro de Ciencias Básicas · Ingeniería en Sistemas Computacionales (9.º semestre)

COMPILADORES II

Fase 3: Análisis Semántico

Compilador e IDE del lenguaje CAOS (Proyecto-IDECOMP)

Integrantes	Matrícula
Oscar Manuel García Rodríguez	280196
Carlos Franco Acosta	283378
Saul Alvarez Gaspar	289460

Docente: Dra. Blanca G. Estrada Rentería
Entrega: 15 de octubre de 2026

Repositorio: github.com/OscarArise/Proyecto-IDECOMP (etiquetas fase3-b1, fase3-b2 y fase3-entrega)

1. Descripción de la implementación

El análisis semántico es la tercera fase del compilador. Recibe el Árbol Sintáctico Abstracto (AST) que produce el analizador sintáctico, calcula atributos de tipo y de valor sobre una **copia** del árbol, construye la tabla de símbolos y reporta los errores semánticos. El AST original no se modifica, de modo que ambos árboles pueden compararse dentro del IDE.

El trabajo se dividió en tres bloques: (1) preparación del analizador sintáctico y del contrato del AST, (2) núcleo semántico (tabla de símbolos, reglas, atributos y errores) y (3) integración con el compilador y el IDE, evidencias y este documento.

Pipeline del compilador

```
fuente → léxico (DFA) → sintáctico (descenso recursivo) → AST
     → semántico: copia anotada + tabla de símbolos + errores
     → (intermedio y ejecución: fases posteriores)

Códigos de salida:  0 sin errores · 1 léxico · 2 sintáctico · 3 semántico
```

Módulos

Archivo	Responsabilidad
semantic/tipos.py	Tipos int, float y bool; grupos de operadores; reglas de compatibilidad y conversión.
semantic/errores.py	Códigos E1–E7 y la clase ErrorSemantico ([línea:columna] Error semántico Ex: mensaje).
semantic/tabla_simbolos.py	Tabla hash con encadenamiento (insertar, buscar, actualizar líneas, eliminar).
semantic/analizador.py	Recorrido del AST, atributos heredados y sintetizados, verificación de tipos, conversiones y anotación.
semantic/formateador.py	Genera semantic.txt, symbols.txt y las líneas de errores.
compiler_stub.py	Conecta la fase al pipeline: recibe el AST y escribe los archivos de salida.
ide/ui/*	Pestañas Semantico, Arbol Anotado, Comparacion, Simbolos y errores semánticos; marcas en el editor.

2. Reglas semánticas utilizadas

El lenguaje tiene un solo ámbito (el bloque **main**) y tres tipos: **int**, **float** y **bool**. La palabra **real** es un alias de float; las cadenas solo aparecen dentro de **cout** y no participan en la verificación de tipos.

Construcción	Regla semántica
Declaración tipo id1, id2, ...	dtype ← tipo (heredado); cada idi se inserta en la tabla con ese tipo. Si ya existía: E2.
Identificador en expresión o asignación	Se busca en la tabla; tipo ← tipo registrado y valor ← último valor conocido de la variable. Si no está: E1.
Literal	tipo ← int o float según el literal; valor ← el literal; true/false: bool.
a + b, a − b, a * b, a / b, a ^ b	Si ambos son int, el resultado es int; si alguno es float, el resultado es float y el operando int se convierte. Con bool: E5.
a % b	Solo int con int; el resultado es int. En otro caso: E5.
a < b, a <= b, a > b, a >= b	Operandos numéricos; resultado bool. En otro caso: E5.
a == b, a != b	Numérico con numérico, o bool con bool; resultado bool. En otro caso: E3.
a && b, a || b, !a	Operandos bool; resultado bool. En otro caso: E5.
id = expresión	Mismo tipo: compatible. float ← int: conversión implícita. int ← float: E4. bool contra número: E6. Si es válida, el valor de la expresión se guarda en la tabla como valor actual de id.
id++, id−−	id debe ser numérico; en otro caso: E5. Su valor aumenta o disminuye en 1.
cin id	El valor se conoce hasta la ejecución: el valor de id queda desconocido.
Condición de if, while y until (do-until)	La expresión debe ser bool; en otro caso: E7.

División entre enteros. int / int produce int y trunca hacia cero, como en C. La guía solo pide conversión cuando los operandos son de tipos diferentes; con dos int no hay conversión. Con división real, las líneas 11, 12, 13 y 31 del archivo de prueba producirían cuatro errores E4 que el archivo no busca provocar.

Errores en cascada. Un nodo con error recibe el tipo interno error; una operación con un operando de tipo error no genera errores nuevos. Por ejemplo, x = (y + 1.5) * 2 con y no declarada produce un solo E1.

Valores de las variables. Cada variable guarda en la tabla su último valor conocido; un identificador dentro de una expresión toma ese valor (en z = y + 7, y vale 4 y z recibe 11). Como pide la guía, el valor no se fuerza cuando no puede conocerse en compilación: una variable sin asignar o leída con cin queda desconocida. En un if con condición conocida solo la rama que se ejecuta cambia valores; la otra se verifica igual (tipos y errores) y se marca como no se ejecuta. Si la condición no se conoce, se analizan las dos ramas y una variable conserva su valor solo si coincide en ambas. El cuerpo de un ciclo se anota con los valores de la primera iteración; como no se sabe cuántas veces se repite, las variables que cambian dentro del ciclo quedan desconocidas al salir, salvo que el while no se ejecute o el until se cumpla en la primera pasada.

3. Atributos sintetizados y heredados

Atributo	Clase	Dónde se calcula	Cómo se propaga
dtype	Heredado	Nodo de declaración (tipo)	De la declaración hacia sus identificadores (preorden); es el tipo con el que cada identificador entra en la tabla.
tipo	Sintetizado	Literales, identificadores y operaciones	De los hijos al padre (postorden), según las reglas de la sección 2.
valor	Sintetizado	Literales, identificadores, operaciones y asignaciones	De los hijos al padre. Un identificador toma su valor de la tabla; una asignación guarda el valor en la tabla. Si algún operando es desconocido no se fuerza; una división entre cero no se pliega.
loc	De tabla	Identificadores	Posición del símbolo en la tabla, en orden de declaración (0, 1, 2…).
tipo de la expresión / verificación	Sintetizado	Asignaciones	Resultado de comparar el tipo de la variable con el de la expresión: compatible, conversión o error.
tipo requerido	Heredado	Condiciones	Las condiciones de if, while y until heredan el tipo requerido bool.

4. Anotación del AST

El analizador trabaja sobre una copia profunda del árbol; el árbol original queda intacto (hay una prueba automática que lo verifica). A cada nodo de la copia se le agregan claves y su etiqueta se reescribe con las anotaciones entre ⟨ ⟩, de modo que el visor del IDE las muestra sin cambios.

Clave agregada	Nodos	Contenido
tipo_dato, atributo_tipo	Expresiones, identificadores	Tipo calculado y su clase (sintetizado o heredado).
valor_calculado	Expresiones e identificadores	Valor obtenido en compilación, cuando se conoce.
valor_asignado	Asignaciones, ++ y −−	Valor que se guarda en la tabla.
ejecucion	Ramas o cuerpos que no se ejecutan	Se verifican, pero no cambian los valores.
dtype, loc	Declaraciones e identificadores	Tipo heredado y localidad en la tabla.
tipo_expresion, verificacion	Asignaciones	Tipo de la expresión y resultado: compatible, conversion o error.
tipo_requerido	Secciones CONDICION*	Siempre bool.
errores_semanticos	Nodo que causó el error	Lista de códigos E1–E7.
label_sintactico	Nodos anotados	Etiqueta original del analizador sintáctico.

Se agregan dos tipos de nodo: **IdentificadorDeclarado** (hijo de cada declaración, uno por variable, con su dtype heredado y su loc) y **ConversionTipo** (envuelve al operando que se

convirtió de int a float).

Fragmento del árbol anotado (líneas 2 a 9 del archivo de prueba)

Los identificadores toman su valor de la tabla: en la línea 8 y vale 4 y en la línea 9 se usa ese valor para calcular y = y + 1 = 5.

```
    ID: x  ⟨dtype: int (heredado), loc: 0⟩ [L2:C9]
    ID: y  ⟨dtype: int (heredado), loc: 1⟩ [L2:C12]
    ID: z  ⟨dtype: int (heredado), loc: 2⟩ [L2:C15]
  DECLARACION_VARIABLE: float a, b, c  ⟨dtype: float⟩ [L3:C5]
    ID: a  ⟨dtype: float (heredado), loc: 3⟩ [L3:C11]
    ID: b  ⟨dtype: float (heredado), loc: 4⟩ [L3:C14]
    ID: c  ⟨dtype: float (heredado), loc: 5⟩ [L3:C17]
  BLOQUE [L4:C5]
    ASIGNACION: suma  ⟨destino no declarado, ✗ E1⟩ [L4:C5]
      NUMERO: 45  ⟨tipo: int, valor: 45⟩ [L4:C12]
    ASIGNACION: x  ⟨x: int, expresión: float, ✗ E4⟩ [L5:C5]
      NUMERO: 32.32  ⟨tipo: float, valor: 32.32⟩ [L5:C9]
    ASIGNACION: x  ⟨x: int, expresión: int, valor: 23⟩ [L6:C5]
      NUMERO: 23  ⟨tipo: int, valor: 23⟩ [L6:C9]
    ASIGNACION: y  ⟨y: int, expresión: int, valor: 4⟩ [L7:C5]
      OPERACION: -  ⟨tipo: int, valor: 4⟩ [L7:C15]
        OPERACION: +  ⟨tipo: int, valor: 5⟩ [L7:C11]
          NUMERO: 2  ⟨tipo: int, valor: 2⟩ [L7:C9]
          NUMERO: 3  ⟨tipo: int, valor: 3⟩ [L7:C13]
        NUMERO: 1  ⟨tipo: int, valor: 1⟩ [L7:C17]
    ASIGNACION: z  ⟨z: int, expresión: int, valor: 11⟩ [L8:C5]
      OPERACION: +  ⟨tipo: int, valor: 11⟩ [L8:C11]
        IDENTIFICADOR: y  ⟨tipo: int, valor: 4, loc: 1⟩ [L8:C9]
        NUMERO: 7  ⟨tipo: int, valor: 7⟩ [L8:C13]
    ASIGNACION: y  ⟨y: int, expresión: int, valor: 5⟩ [L9:C5]
      OPERACION: +  ⟨tipo: int, valor: 5⟩ [L9:C11]
        IDENTIFICADOR: y  ⟨tipo: int, valor: 4, loc: 1⟩ [L9:C9]
        NUMERO: 1  ⟨tipo: int, valor: 1⟩ [L9:C13]
…
```

Fragmento del árbol anotado: ciclos anidados (líneas 28 a 38)

El while de la línea 30 es un ciclo anidado dentro del cuerpo del do-until. Su condición x > 7 vale false (x = 7), así que su cuerpo se verifica pero no cambia valores.

```
  DO_UNTIL  ⟨número de iteraciones desconocido⟩ [L28:C5]
    CUERPO  ⟨valores de la primera iteración⟩ [L29:C9]
      BLOQUE [L29:C9]
        ASIGNACION: y  ⟨y: int, expresión: int, valor: 1⟩ [L29:C9]
          OPERACION: +  ⟨tipo: int, valor: 1⟩ [L29:C25]
            OPERACION: *  ⟨tipo: int, valor: 0⟩ [L29:C21]
              OPERACION: +  ⟨tipo: int, valor: 0⟩ [L29:C16]
                IDENTIFICADOR: y  ⟨tipo: int, valor: -1, loc: 1⟩ [L29:C14]
                NUMERO: 1  ⟨tipo: int, valor: 1⟩ [L29:C18]
              NUMERO: 2  ⟨tipo: int, valor: 2⟩ [L29:C23]
            NUMERO: 1  ⟨tipo: int, valor: 1⟩ [L29:C27]
        WHILE  ⟨condición inicial false → el cuerpo no se ejecuta⟩ [L30:C9]
          CONDICION  ⟨requiere: bool, obtenido: bool, valor: false⟩ [L30:C17]
            OPERACION: >  ⟨tipo: bool, valor: false⟩ [L30:C17]
              IDENTIFICADOR: x  ⟨tipo: int, valor: 7, loc: 0⟩ [L30:C15]
              NUMERO: 7  ⟨tipo: int, valor: 7⟩ [L30:C19]
          CUERPO  ⟨no se ejecuta: la condición del while es false al llegar al ciclo⟩ [L31:C13]
            BLOQUE [L31:C13]
              ASIGNACION: x  ⟨x: int, expresión: int, valor: 6⟩ [L31:C13]
                …
              ENTRADA: cin x  ⟨tipo: int, valor: se lee en ejecución⟩ [L32:C13]
              ASIGNACION: mas  ⟨destino no declarado, ✗ E1⟩ [L33:C13]
                OPERACION: /  ⟨tipo: int, valor: 5⟩ [L33:C22]
                  NUMERO: 36  ⟨tipo: int, valor: 36⟩ [L33:C19]
                  NUMERO: 7  ⟨tipo: int, valor: 7⟩ [L33:C24]
    CONDICION_UNTIL  ⟨requiere: bool, obtenido: bool, valor: false⟩ [L35:C13]
      OPERACION: ==  ⟨tipo: bool, valor: false⟩ [L35:C13]
        IDENTIFICADOR: y  ⟨tipo: int, valor: 1, loc: 1⟩ [L35:C11]
```

```
      NUMERO: 5  (tipo: int, valor: 5) [L35:C16]
  WHILE  (número de iteraciones desconocido) [L36:C5]
    CONDICION  (requiere: bool, obtenido: bool, valor: desconocido) [L36:C13]
      OPERACION: ==  (tipo: bool) [L36:C13]
        IDENTIFICADOR: y  (tipo: int, valor: desconocido, loc: 1) [L36:C11]
        NUMERO: 0  (tipo: int, valor: 0) [L36:C16]
    CUERPO  (valores de la primera iteración) [L37:C9]
      BLOQUE [L37:C9]
        ENTRADA: cin mas  (no declarado, ✗ E1) [L37:C9]
        SALIDA: cout [L38:C9]
          SALIDA [L38:C14]
            IDENTIFICADOR: x  (tipo: int, valor: 7, loc: 0) [L38:C14]
```

5. Diseño y funcionamiento de la tabla de símbolos

Es la tabla hash de TINY: tamaño 211, función hash `temp = ((temp << 4) + c) mod 211` sobre los caracteres del nombre, y encadenamiento separado para las colisiones. Se extendió con el tipo y la posición de la declaración.

Campo	Contenido
nombre	Lexema del identificador.
tipo	int, float o bool (viene del atributo heredado dtype).
loc	Desplazamiento de memoria: 0, 1, 2... en orden de declaración.
línea y columna de declaración	Dónde se declaró.
líneas	Una línea por cada aparición (declaración y cada uso). Si el identificador aparece dos veces en la misma línea, el número se repite (y = y + 1 en la línea 9 registra 9, 9), igual que st_insert de TINY.
valor	Último valor conocido en compilación, o desconocido.
siguiente	Siguiente símbolo de la misma cubeta (encadenamiento).

Operaciones: **insertar** (si existe, se reporta E2), **buscar** (si no existe, E1; si existe, agrega la línea de uso), **actualizar líneas**, **asignar valor** y **eliminar**. Las variables no declaradas no se insertan: se reporta E1 en cada uso. Como el recorrido sigue el orden del programa, usar una variable antes de declararla también es E1.

Tabla generada con el archivo de prueba

```
Tabla de Símbolos
================================================
Tabla hash con encadenamiento (SIZE = 211, función hash de TINY).
Ámbito único: bloque main.
Líneas: una por cada aparición del identificador (se repite si aparece
varias veces en la misma línea). Valor final: último valor conocido en
compilación ('desconocido' si depende de la ejecución).

Nombre  Tipo   Loc  Hash  Declarada  Valor final  Líneas
------  -----  ---  ----  ---------  -----------  --------------------------------------------------
x       int    0    120   2:9        7            2, 5, 6, 11, 27, 30, 31, 32, 38
y       int    1    121   2:12       desconocido  2, 7, 8, 9, 9, 12, 14, 16, 23, 23, 29, 29, 35, 36
z       int    2    122   2:15       64           2, 8, 13
a       float  3    97    3:11       62.0         3, 10, 16, 25
b       float  4    98    3:14       3.2          3, 19, 21
c       float  5    99    3:17       desconocido  3, 26

Total de identificadores: 6
```

6. Comprobación de tipos y conversiones

La comprobación se hace en postorden: primero se determina el tipo de los hijos y después el del padre. Con el archivo de prueba el analizador aplicó **5 conversiones implícitas** int → float; todas ocurren en la línea 10 (a = 24.0 + 4 − 1 / 3 * 2 + 34 − 1), donde un operando int se combina con float:

```
[10:12] CONVERSION: int → float sobre 'NUMERO: 4'
[10:17] CONVERSION: int → float sobre 'OPERACION: *'
[10:20] CONVERSION: int → float sobre 'NUMERO: 34'
[10:23] CONVERSION: int → float sobre 'NUMERO: 1'
[16:17] CONVERSION: int → float sobre 'NUMERO: 3'
```

La única conversión implícita permitida es **int → float**, al combinar un int con un float o al asignar un int a una variable float. float → int y cualquier conversión con bool no son válidas: se reportan como E4 (y E6 cuando se mezcla bool con número).

Recorrido semántico (fragmento)

Cada paso queda registrado en semantic.txt: declaraciones y herencia de dtype, inserciones y búsquedas en la tabla, tipos obtenidos, conversiones y errores.

```
[2:5] DECLARACION: dtype = int se hereda a: x, y, z
[2:9] INSERTAR: x → tipo int, loc 0, hash 120
[2:12] INSERTAR: y → tipo int, loc 1, hash 121
[2:15] INSERTAR: z → tipo int, loc 2, hash 122
[3:5] DECLARACION: dtype = float se hereda a: a, b, c
[3:11] INSERTAR: a → tipo float, loc 3, hash 97
[3:14] INSERTAR: b → tipo float, loc 4, hash 98
[3:17] INSERTAR: c → tipo float, loc 5, hash 99
[4:5] BUSCAR: suma → no está en la tabla
[4:5] ERROR E1: 'suma' no está declarada
[5:5] BUSCAR: x → tipo int, loc 0, valor desconocido
[5:5] ERROR E4: No se puede asignar un valor float a 'x', que es int (la conversión float → int no está
    permitida)
[6:5] BUSCAR: x → tipo int, loc 0, valor desconocido
[6:5] ASIGNACION: x: int ← int (compatible)
[6:5] VALOR: x ← 23
[7:5] BUSCAR: y → tipo int, loc 1, valor desconocido
[7:11] TIPO: int + int → int, valor 5
[7:15] TIPO: int - int → int, valor 4
[7:5] ASIGNACION: y: int ← int (compatible)
[7:5] VALOR: y ← 4
[8:5] BUSCAR: z → tipo int, loc 2, valor desconocido
[8:9] BUSCAR: y → tipo int, loc 1, valor 4
[8:11] TIPO: int + int → int, valor 11
[8:5] ASIGNACION: z: int ← int (compatible)
```

7. Errores semánticos implementados

Código	Error	Se produce cuando…
E1	Identificador no declarado	Se usa una variable que no está en la tabla.
E2	Declaración duplicada	Se declara dos veces el mismo identificador.
E3	Tipos incompatibles	Se comparan con == o != tipos que no son compatibles.
E4	Conversión no válida	Se asigna un float a un int (float → int no está permitida).
E5	Operador mal aplicado	El operador no admite el tipo de sus operandos (&&, %, ++…).
E6	Asignación inconsistente	Se asigna un bool a un número o un número a un bool.
E7	Condición no booleana	if, while o until reciben una expresión que no es bool.

Formato: `[SEMANTICO] [línea:columna] Error semántico Ex: mensaje`. Los errores se escriben en errors.txt (el compilador termina con código 3) y el IDE los muestra en la pestaña Errores Semanticos y los marca en el editor.

Errores detectados en el archivo de prueba

```
[SEMANTICO] [4:5] Error semántico E1: 'suma' no está declarada
[SEMANTICO] [5:5] Error semántico E4: No se puede asignar un valor float a 'x', que es int (la
    conversión float → int no está permitida)
[SEMANTICO] [14:5] Error semántico E4: No se puede asignar un valor float a 'y', que es int (la
    conversión float → int no está permitida)
[SEMANTICO] [16:9] Error semántico E4: No se puede asignar un valor float a 'y', que es int (la
    conversión float → int no está permitida)
[SEMANTICO] [33:13] Error semántico E1: 'mas' no está declarada
[SEMANTICO] [37:13] Error semántico E1: 'mas' no está declarada
```

8. Pruebas realizadas

Archivo de prueba

Se utilizó el archivo TestSemantica.txt del curso. Contiene 40 líneas, declara x, y, z (int) y a, b, c (float) y mezcla asignaciones, expresiones, condicionales, ciclos y entrada/salida. Resultado: **6 identificadores** en la tabla, **5 conversiones** y **6 errores semánticos**.

```
 1  main {
 2      int x, y, z;
 3      float a, b, c;
 4      suma = 45;
 5      x = 32.32;
 6      x = 23;
 7      y = 2 + 3 - 1;
 8      z = y + 7;
 9      y = y + 1;
10      a=24.0+4-1/3*2+34-1;
11      x=(5-3)*(8/2);
12      y=5+3-2*4/7-9;
13      z = 8 / 2 + 15 * 4;
14      y = 14.54;
15      if 2>3 then
16          y = a + 3;
17      else
18          if 4>2 && true then
19              b = 3.2;
20          else
21              b = 5.0;
22          end
23          y = y + 1;
24      end
25      a++;
26      c--;
27      x = 3 + 4;
28      do
29          y = (y + 1) * 2 + 1;
30          while x > 7
31              x = 6 + 8 / 9 * 8 / 3;
32              cin x;
33              mas = 36 / 7;
34          end
35      until y == 5
36      while y == 0
37          cin mas;
38          cout x;
39      end
40  }
```

Ajuste al archivo de prueba. La guía indica que no debe sustituirse el archivo principal de pruebas. No se sustituyó: se ajustaron tres líneas (32, 37 y 38) para que usen la sintaxis de

entrada y salida de nuestra gramática (cin x; y cout x;, sin los operadores >> y << de C++). Las declaraciones, las expresiones y los errores semánticos que deben detectarse son los mismos. El cambio fue consultado y aprobado por la profesora.

Resultados esperados y obtenidos

Línea:col	Error	Causa
4:5	E1	'suma' no está declarada
5:5	E4	x = 32.32: float asignado a int
14:5	E4	y = 14.54: float asignado a int
16:9	E4	y = a + 3: expresión float asignada a int
33:13	E1	'mas' no está declarada
37:13	E1	'mas' no está declarada

El compilador produce exactamente estos seis errores y ninguno más; los errores en cascada están suprimidos.

Pruebas automáticas

Archivo	Pruebas	Qué verifica
test_semantico.py	41	Tabla de símbolos (líneas repetidas), TestSemantica (errores, tabla, valores, conversiones, árbol original intacto, dtype heredado, ciclo anidado), valores de variables (if, while, do-until, cin, ++), casos E1–E7, cascadas y formateadores.
test_integracion.py	8	Compilador completo como subproceso: código de salida 3, los seis errores, tabla, semantic.txt y regresión de los códigos 0/1/2 con archivos de pruebas/ e ide/samples/.
test_parser_recovery.py	20	Analizador sintáctico, incluidos ciclos anidados a cualquier profundidad.
ide/test_semantic_output.py	5	Separación del árbol anotado y del texto en la salida del compilador.

Cambio en el analizador sintáctico

Al probar casos propios se encontró un error que ya existía: el parser rechazaba una declaración posterior a una sentencia (int x; x = 1; int y;) aunque la gramática lo permite. Se corrigió con tres líneas en Parser._declaracion: la lista de sentencias se detiene al encontrar int, float, real o bool. Las pruebas del parser siguen pasando y los códigos de salida de todos los archivos de pruebas/ e ide/samples/ no cambiaron.

Correcciones tras la prerevisión

Observación	Causa	Corrección
El árbol no jala el valor de la variable.	Se había decidido calcular valores solo con constantes; los identificadores nunca tenían valor y la tabla no lo guardaba.	La tabla guarda el valor actual de cada variable; las asignaciones lo actualizan y los identificadores lo toman (sección 2, Valores de las variables).
Los ciclos no se anidan correctamente.	La gramática del parser definía repeticion → do sentencias while cond sentencias until cond: el while de TestSemantica se tomaba como parte del do y el cuerpo del do terminaba en el primer while.	repeticion → do lista_sentencias until expresión. El while es una iteración normal dentro del cuerpo, así que los ciclos se anidan a cualquier profundidad.
La tabla no repite la línea por cada aparición.	agregar_linea descartaba una línea que ya estaba en la lista.	Cada aparición agrega su línea, como en TINY: y registra 9, 9 y 23, 23.

9. Capturas de ejecución en el IDE

Todas las capturas corresponden a la ejecución del IDE (tecla F7) sobre TestSemantica.txt, salvo la Figura 6 (programa válido) y la Figura 8 (regresión sintáctica).

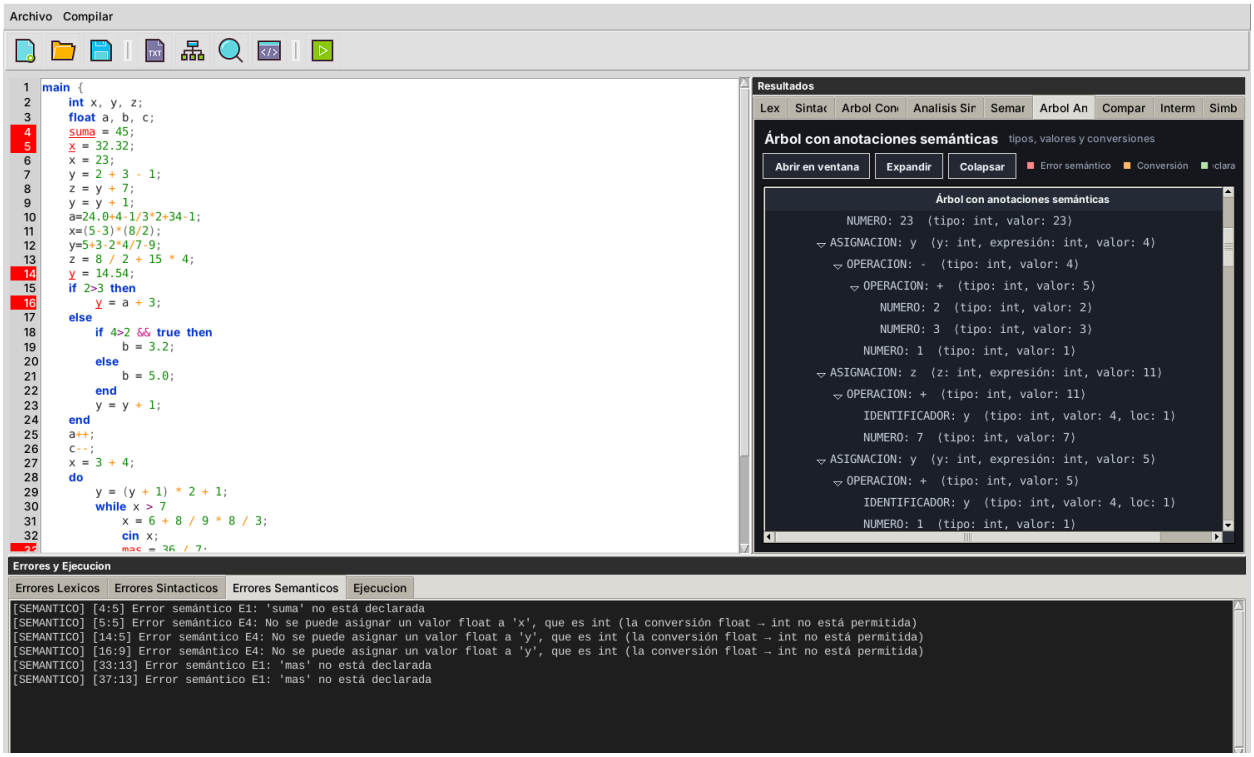


Figura 1. Árbol con anotaciones semánticas: tipos y valores; los identificadores toman su valor de la tabla (en z = y + 7, y vale 4).

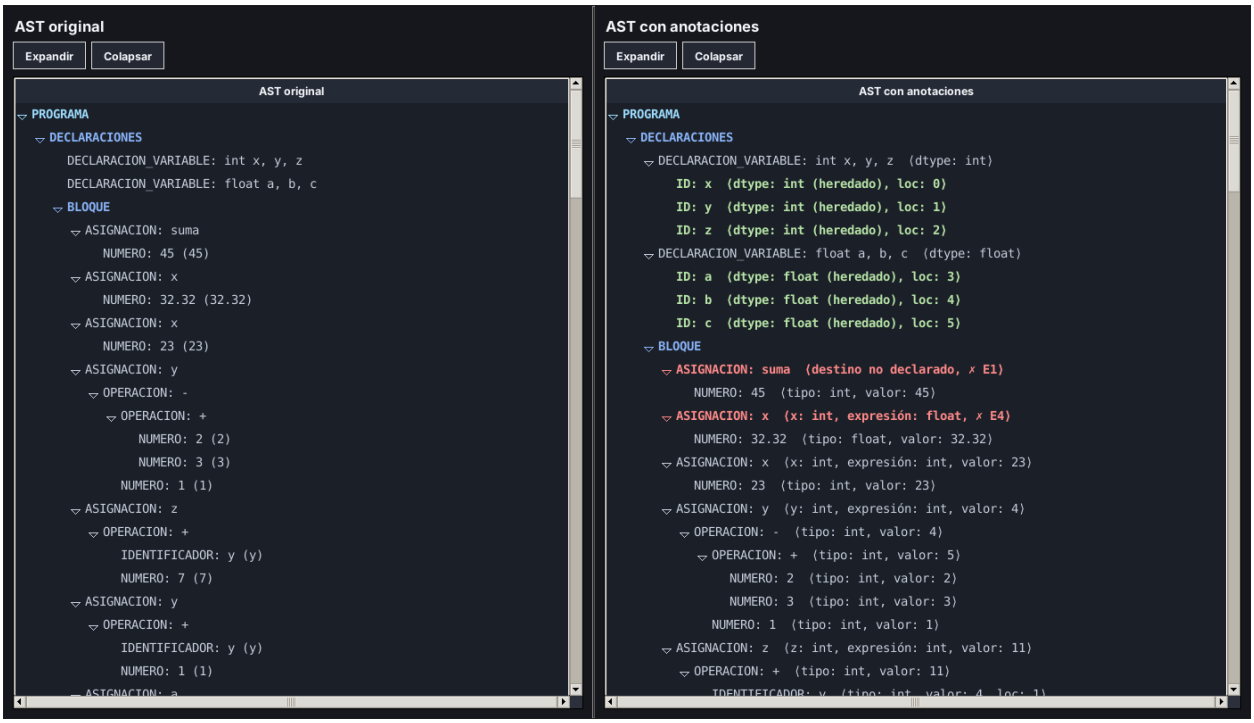


Figura 2. Comparación: AST original (izquierda) frente al AST con anotaciones (derecha).

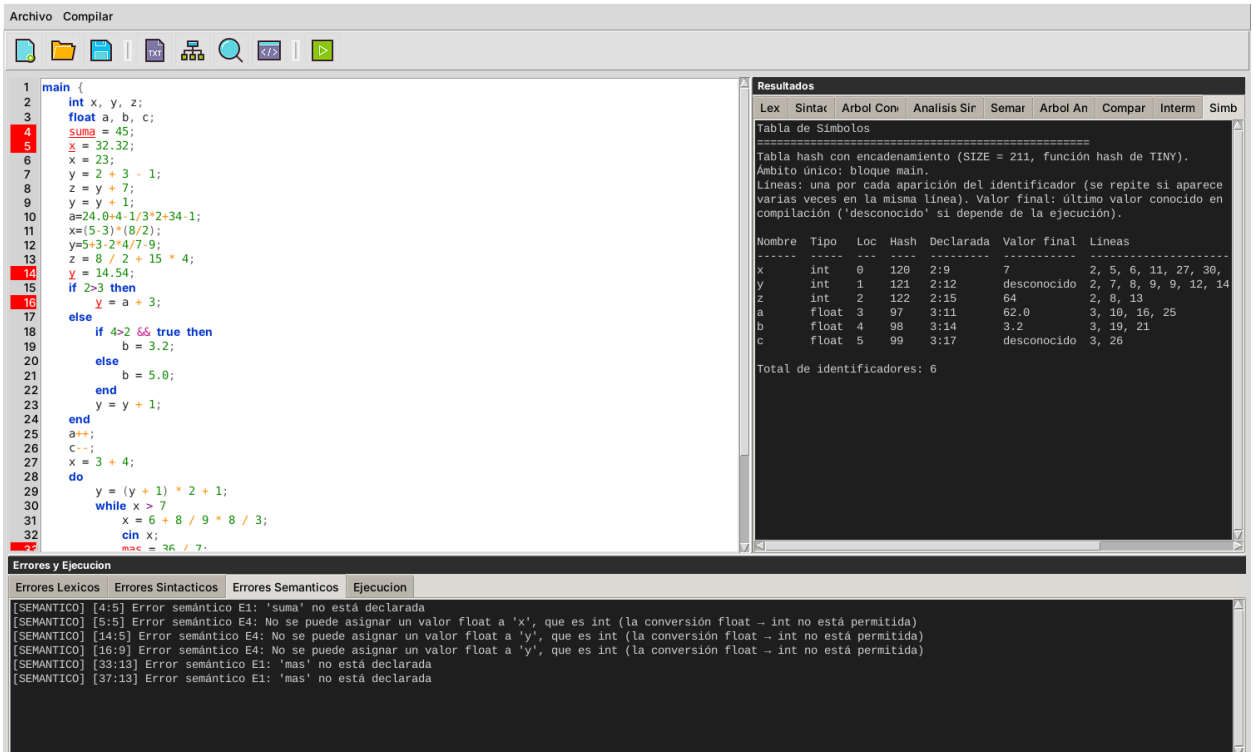


Figura 3. Tabla de símbolos: tipo, localidad, hash, declaración, valor final y una línea por cada aparición.

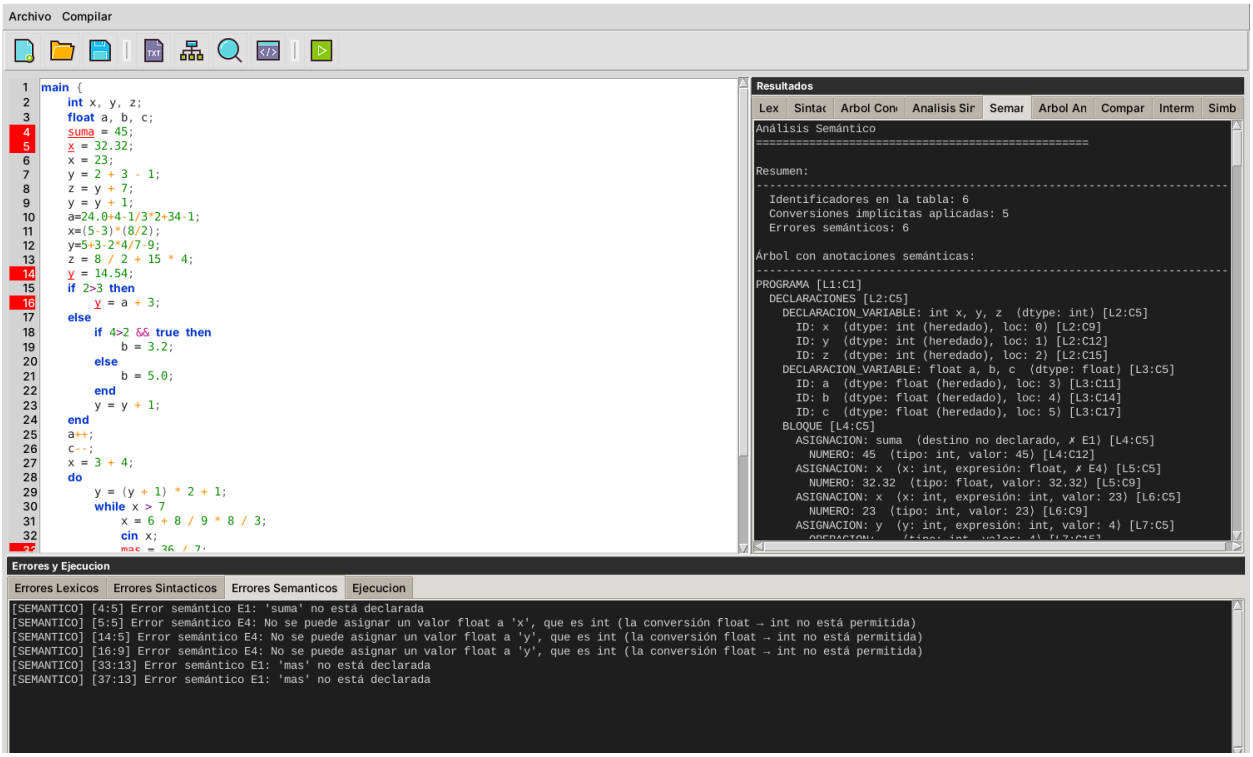


Figura 4. Pestaña Semantico: resumen, árbol anotado en texto y recorrido semántico.

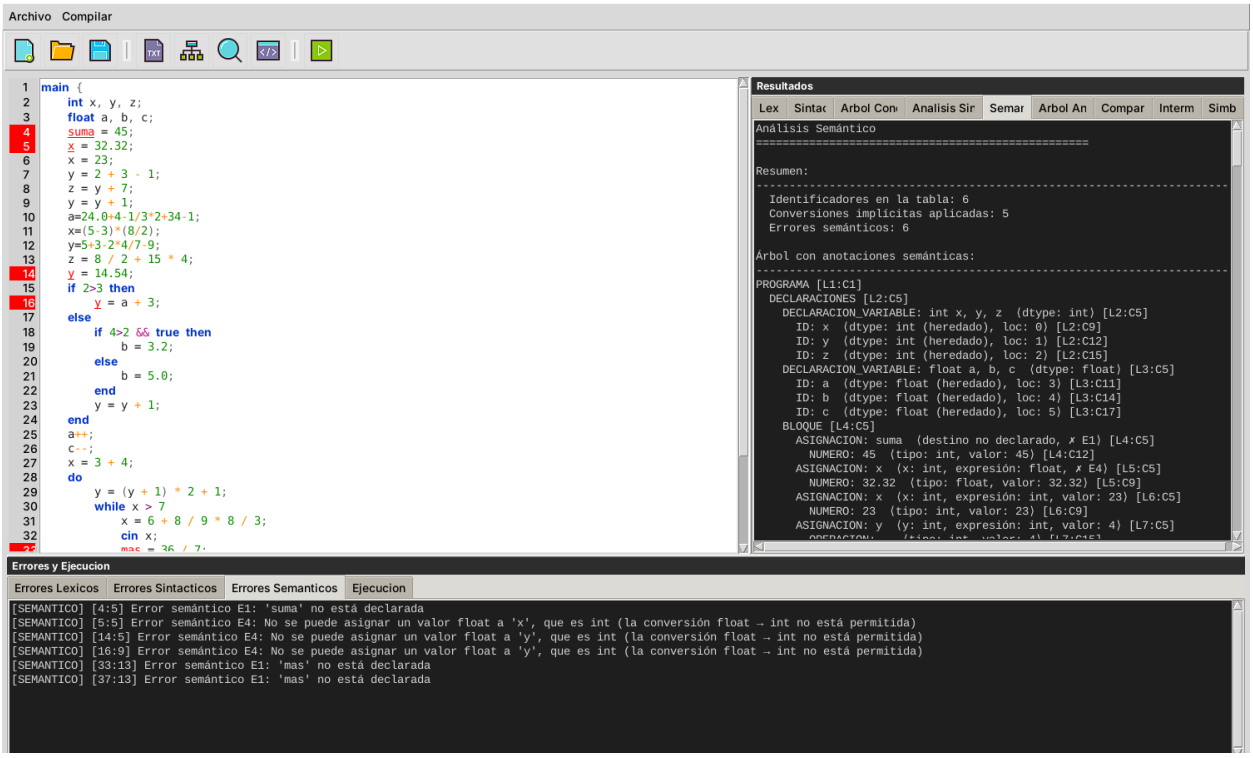


Figura 5. Errores semánticos y marcas en el editor (líneas 4, 5, 14, 16, 33 y 37; identificadores subrayados).

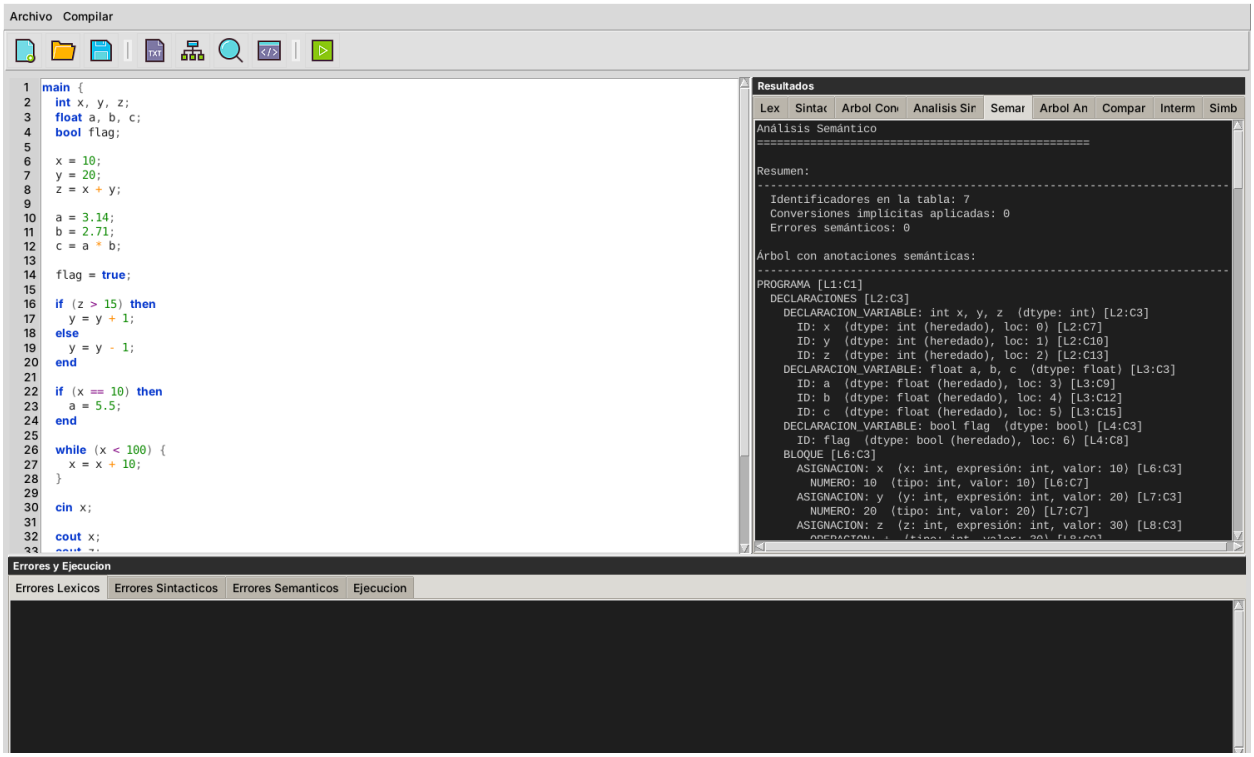


Figura 6. Un programa válido (codigo_valido.caos) completa la fase sin errores.

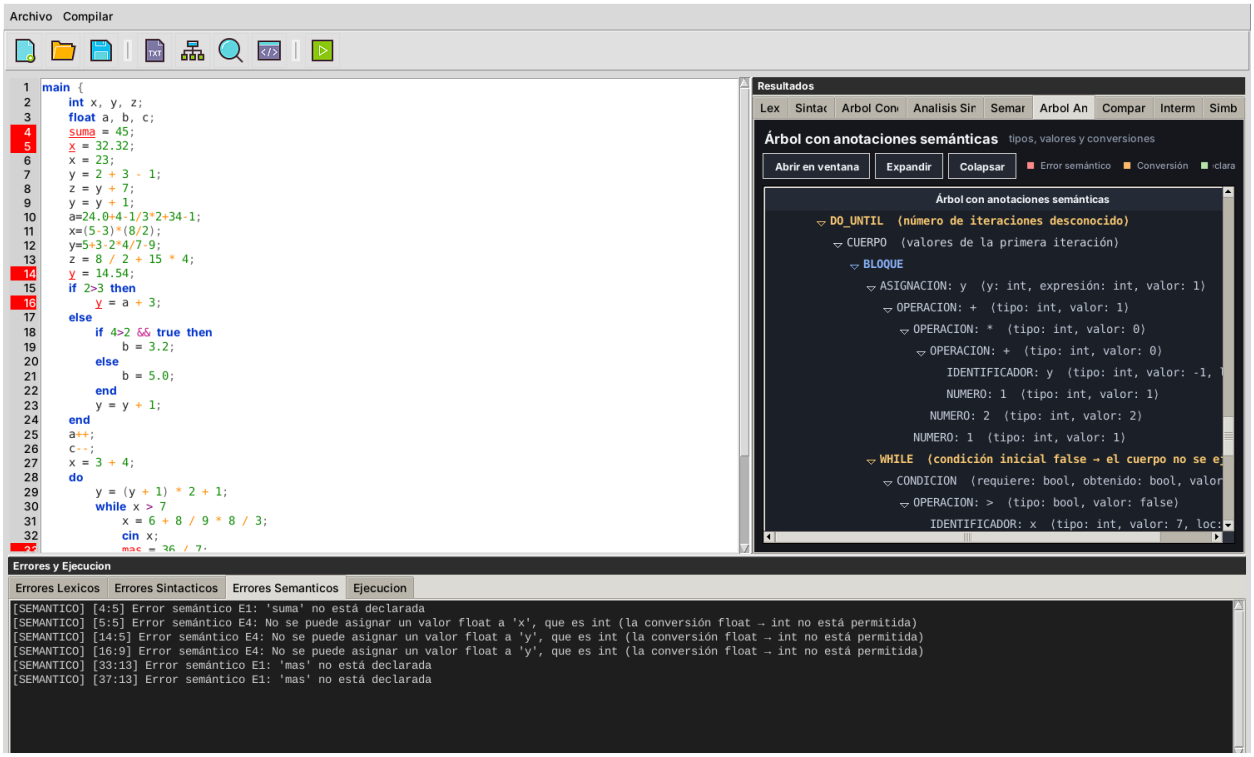


Figura 7. Ciclos anidados: el while de la línea 30 es hijo del cuerpo del do-until; su condición es false, así que su cuerpo no cambia valores.

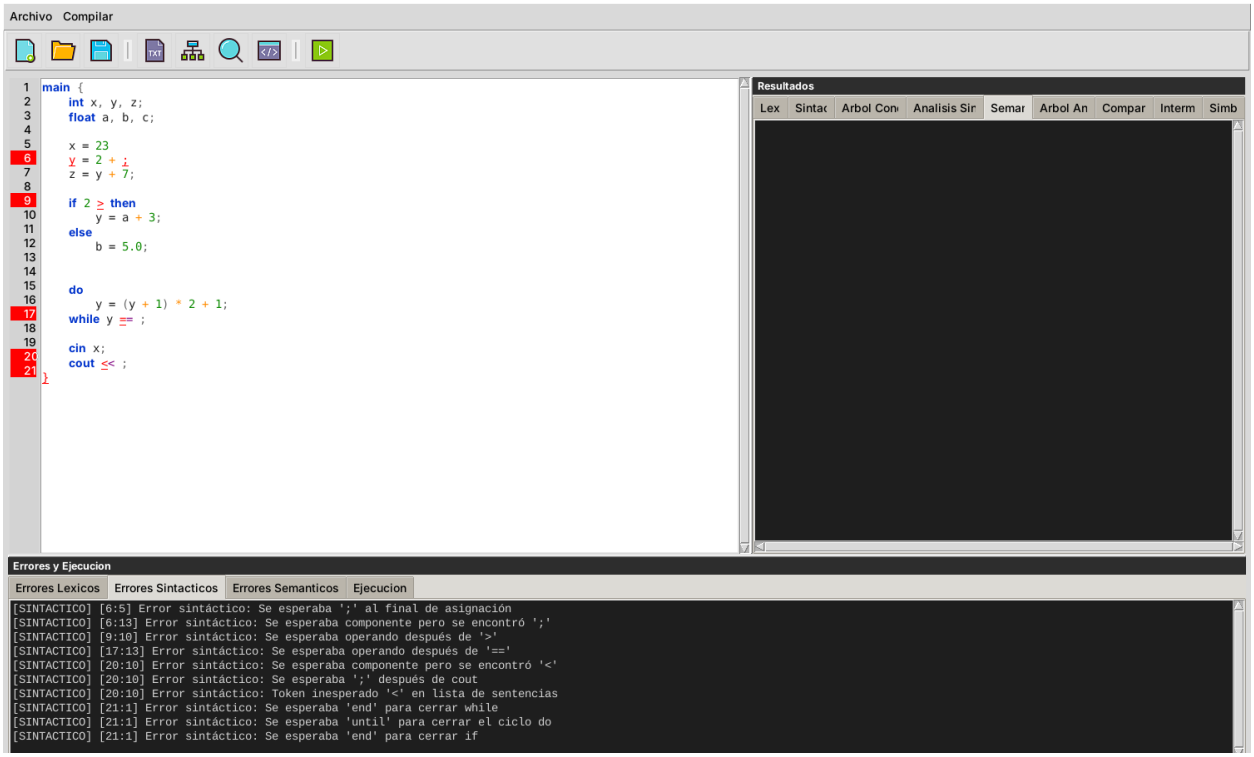


Figura 8. Regresión: un programa con errores sintácticos (conerrores.caos) se detiene en esa fase y los marca en el editor.

10. Entregables

Entregable	Archivo
Código fuente completo	Repositorio (etiqueta fase3-entrega)
AST utilizado como base	ast_original.txt
Árbol con anotaciones semánticas	arbol_anotado.txt
Tabla de símbolos	tabla_simbolos.txt
Errores semánticos	errores_semanticos.txt
Capturas	carpeta de capturas
Documento de la fase	Este PDF
Integrantes y matrículas	integrantes.txt